

3.5.6奇偶校验与可靠性编码(一)

(一)、奇偶校验码

在数码的传送和存储过程中，由于存在干扰，数码可能发生差错。发现这些错误并将它们进行纠正，就是**纠错技术**

1、奇偶校验码

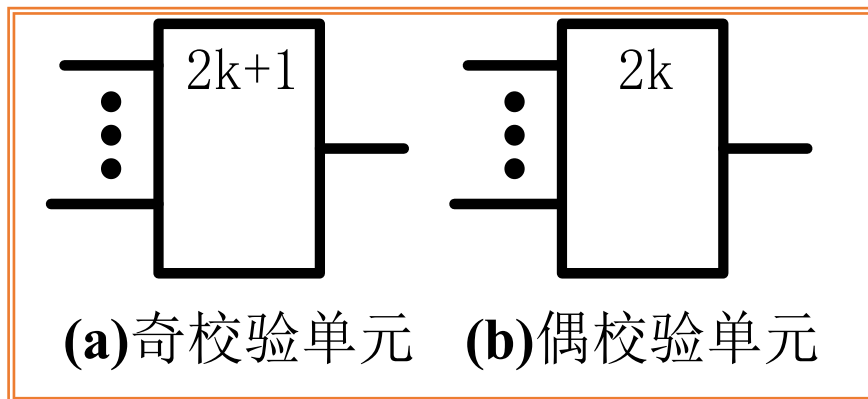
2、汉明码

3.5.6奇偶校验与可靠性编码(二)

(一)、奇偶校验码(续)

奇偶校验码 = 原信息码 + 校验位 监督码元

- **奇校验码：**包含监督码元在内的整个码组中**1的个数为奇数**
若原信息码中的1的个数为奇数，则校验位为0，否则为1
- **偶校验码：**包含监督码元在内的整个码组中**1的个数为偶数**
若原信息码中的1的个数为偶数，则校验位为0，否则为1



奇偶校验单元的通用符号

3.5.6奇偶校验与可靠性编码(三)

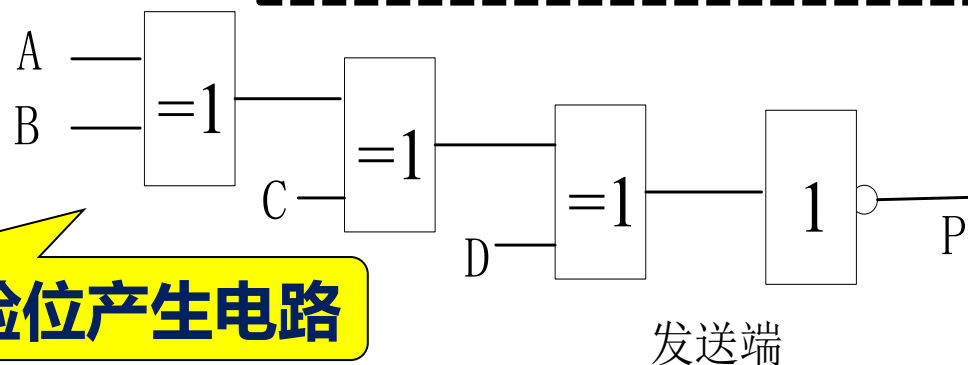
(一)、奇偶校验码(续) 8421BCD码的奇偶校验码

8421BCD码	奇校验位	偶校验位
0 0 0 0	1	0
0 0 0 1	0	1
0 0 1 0	0	1
0 0 1 1	1	0
0 1 0 0	0	1
0 1 0 1	1	0
0 1 1 0	1	0
0 1 1 1	0	1
1 0 0 0	0	1
1 0 0 1	1	0

3.5.6奇偶校验与可靠性编码(四)

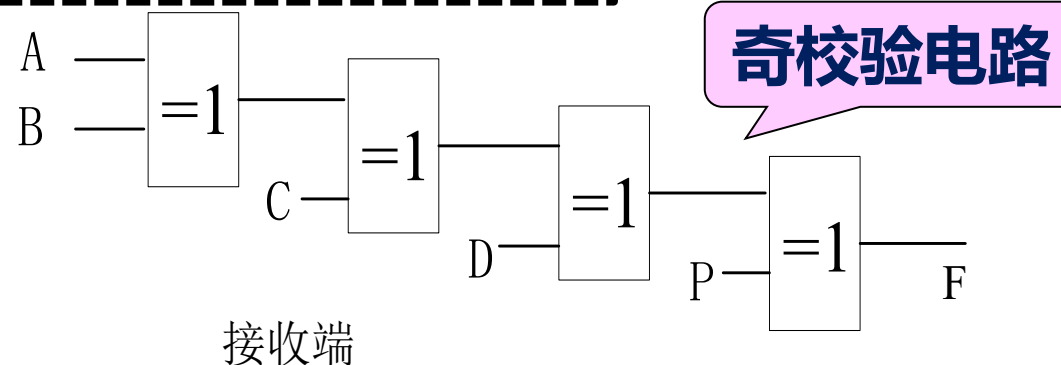
(一)、奇偶校验码(续) 奇校验位的产生与奇校验电路

异或的性质：奇数个1的异或运算结果为1
偶数个1的异或运算结果为0



$$P = A \oplus B \oplus C \oplus D$$

A、B、C、D中有奇数个1，
则P=0，否则P=1

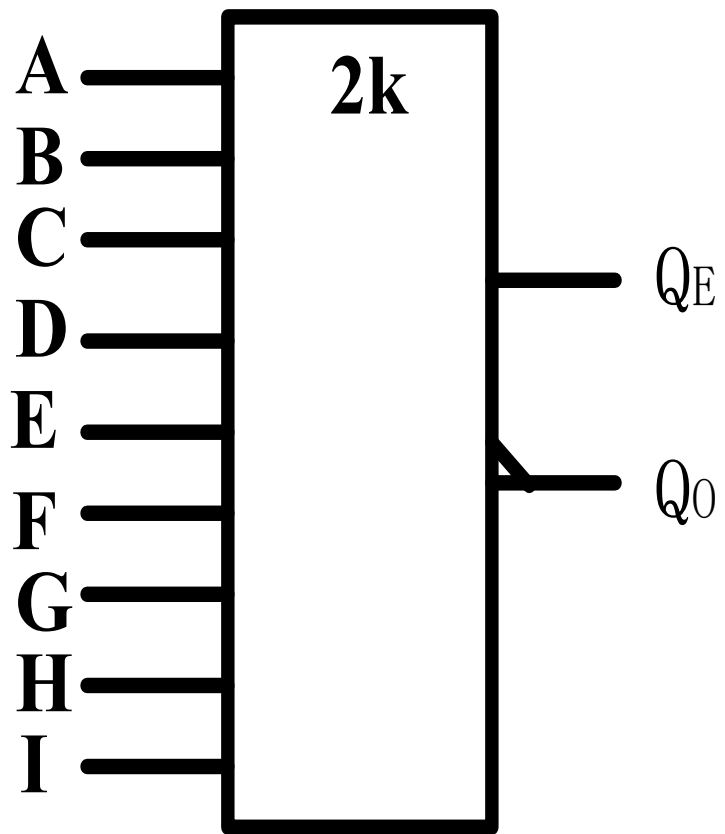


$$F = A \oplus B \oplus C \oplus D \oplus P$$

F=1表示传输正确，否则
传输错误

3.5.6奇偶校验与可靠性编码(五)

(一)、奇偶校验码(续) 集成奇偶校验器74LS280



$$Q_O = A \oplus B \oplus C \oplus D \oplus E \oplus F \oplus G \oplus H \oplus I$$

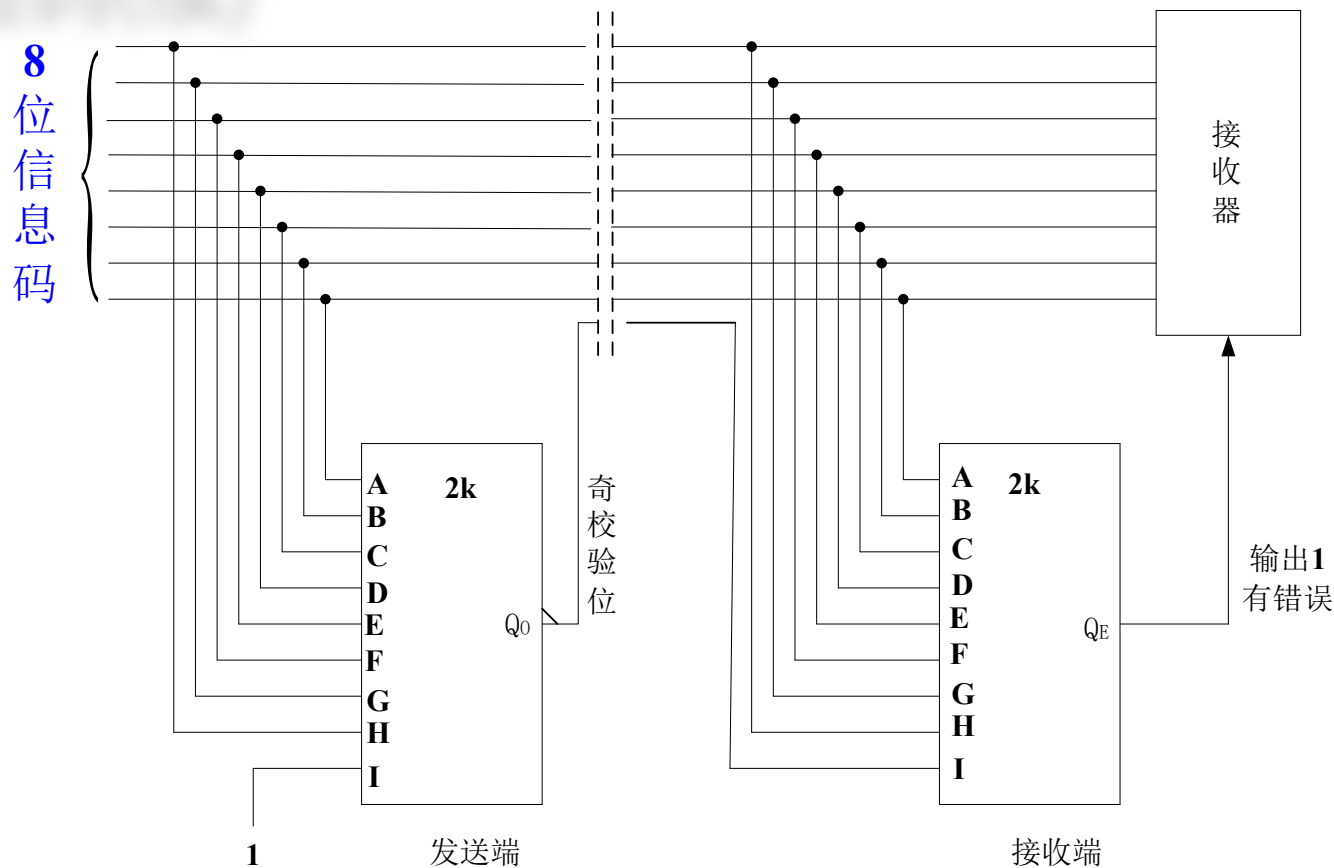
$$Q_E = \overline{A \oplus B \oplus C \oplus D \oplus E \oplus F \oplus G \oplus H \oplus I}$$

A~I中有奇数个1, 则 $Q_O=1$, $Q_E=0$

A~I中有偶数个1, 则 $Q_O=0$, $Q_E=1$

3.5.6奇偶校验与可靠性编码(六)

(一)、奇偶校验码(续) 具有奇偶校验的数据传输



**发送端发送奇校验码，接收端对接收到的码组进行奇校验，
若 $Q_E=0$ 则传输正确，反之则传输错误。**

3.5.6奇偶校验与可靠性编码(七)

(一)、奇偶校验码(续)

奇偶校验的缺点

1、只能检测出1位错，不能检测两位同时出错的情况

两位同时出错的概率很小

2、只能检测出有1位错，但不能确定是哪一位的错，因此不能纠错

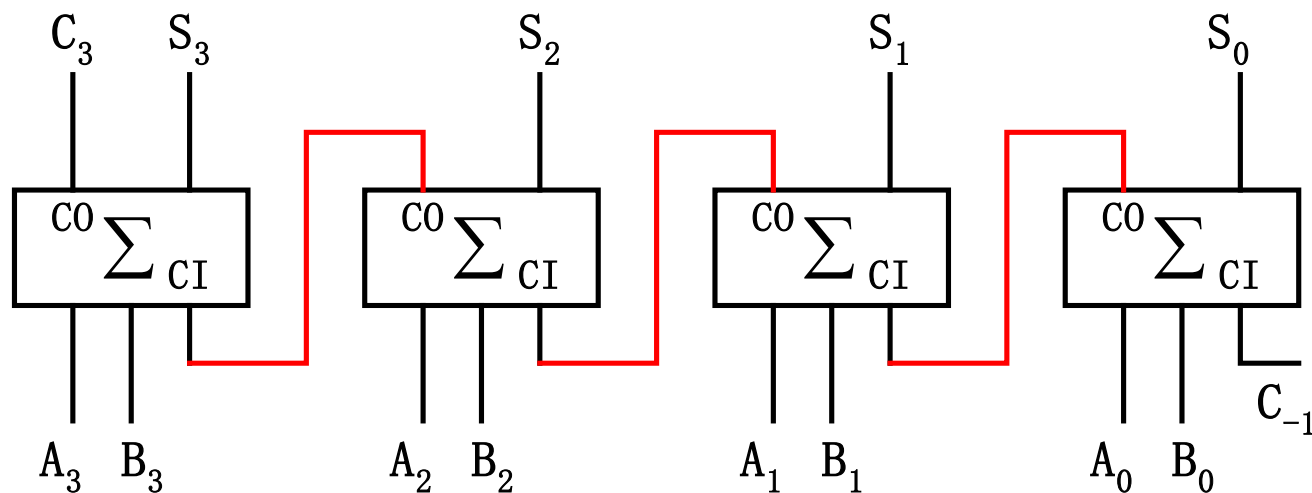
3.5.7 运算电路(一)

(一)、加法器

(1) 串行进位加法器

$$S_i = A_i \oplus B_i \oplus C_{i-1}$$

$$C_i = (A_i \oplus B_i)C_{i-1} + A_i B_i$$



串行进位加法器的缺点：虽然各位相加是并行的，但其**进位信号是由低位向高位逐级传递的**，因此运算速度较慢。

3.5.7 运算电路(二)

(一)、加法器(续)

(2) 超前进位加法器

$$S_i = A_i \oplus B_i \oplus C_{i-1}$$

$$C_i = (A_i \oplus B_i)C_{i-1} + A_i B_i$$

作代换:

$$G_i = A_i B_i$$

$$P_i = (A_i \oplus B_i)$$

$$C_i = G_i + P_i C_{i-1}$$

$$C_1 = G_1 + P_1 C_0$$

三级门电路

$$C_2 = G_2 + P_2 C_1 = G_2 + P_2 G_1 + P_2 P_1 C_0$$

三级门电路

$$C_3 = G_3 + P_3 C_2 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 C_0$$

三级门电路

$$C_4 = G_4 + P_4 C_3 = G_4 + P_4 G_3 + P_4 P_3 G_2 + P_4 P_3 P_2 G_1 + P_4 P_3 P_2 P_1 C_0$$

三级门电路

$$S_i = P_i \oplus C_{i-1}$$

四级门电路

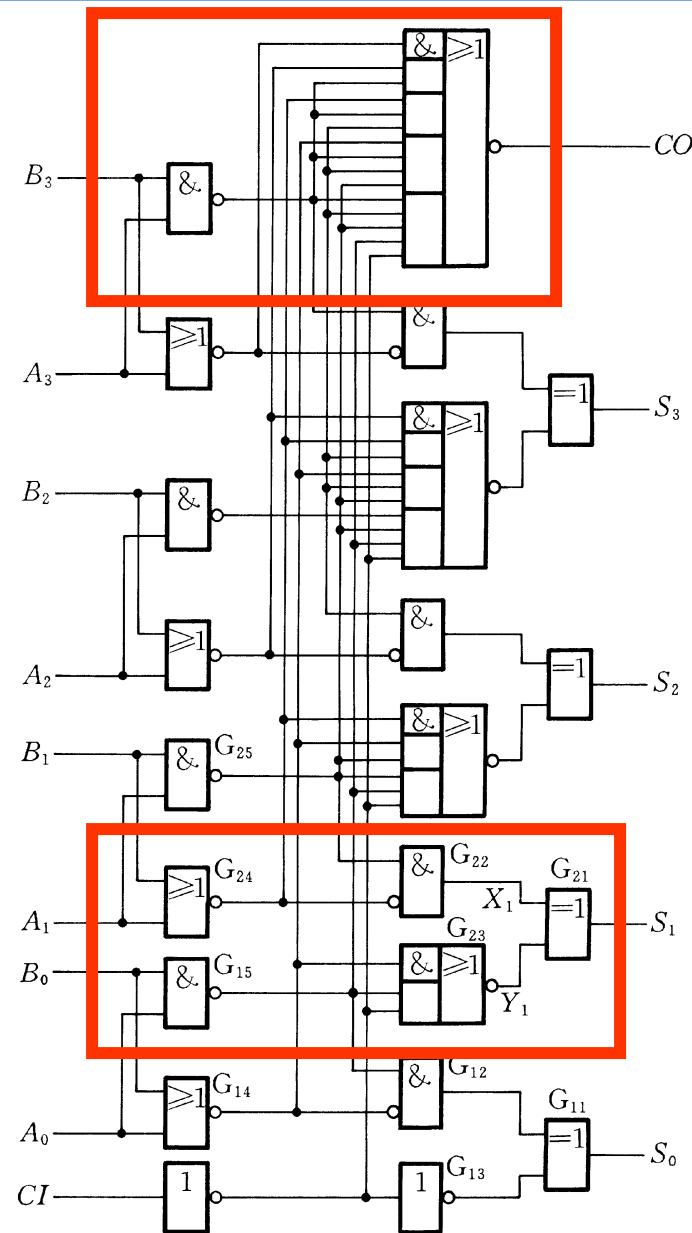
只需四级门电路就可以实现四位加法器

3.5.7 运算电路(三)

(一)、加法器(续)

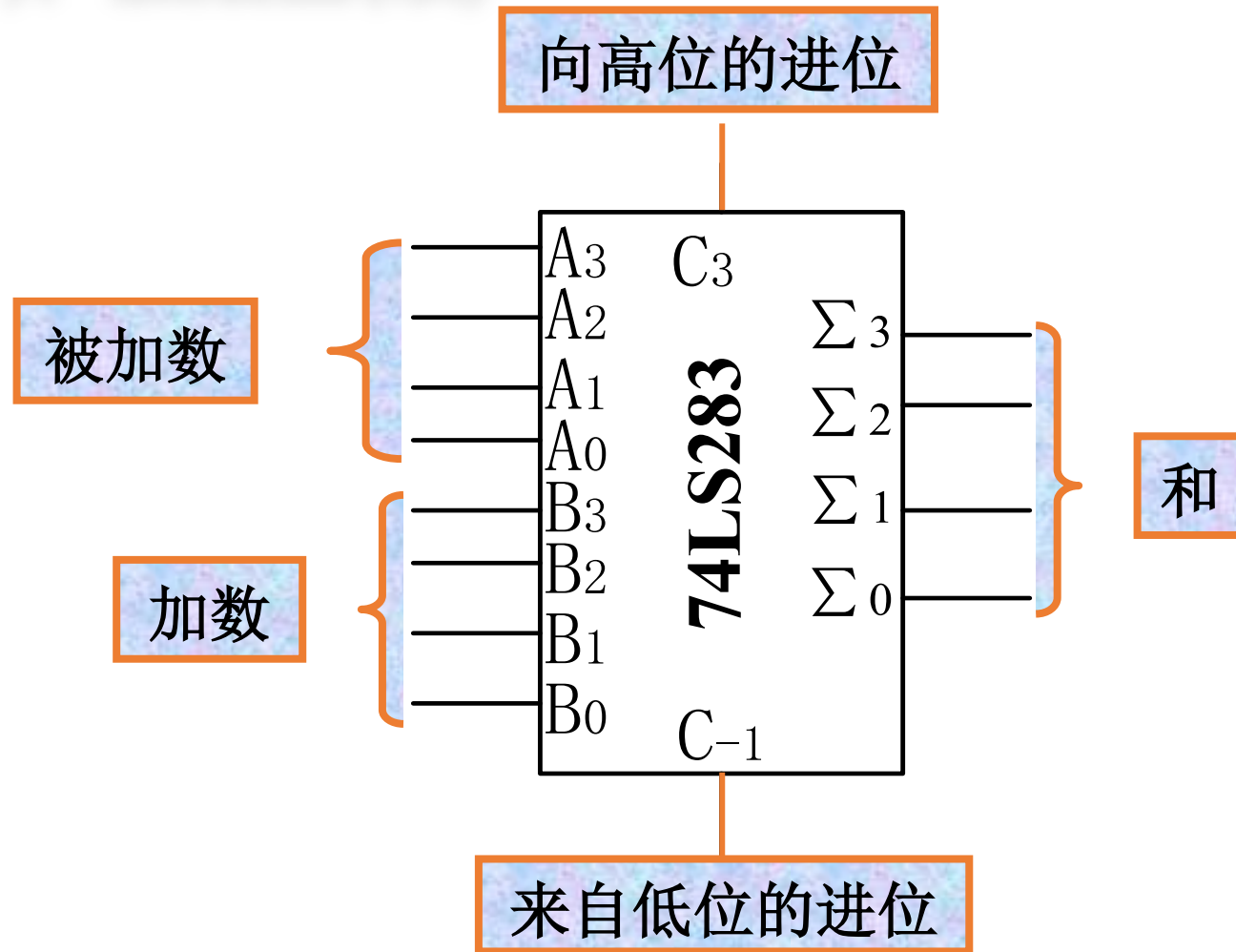
四位超前进位加法器74LS283

- 进位输出信号只有两级门延迟
- 和输出信号只有三级门延迟
- 电路复杂程度的增加
- 随着加法器位数的增多，电路的复杂程度急剧上升



3.5.7 运算电路(四)

(一)、加法器(续)



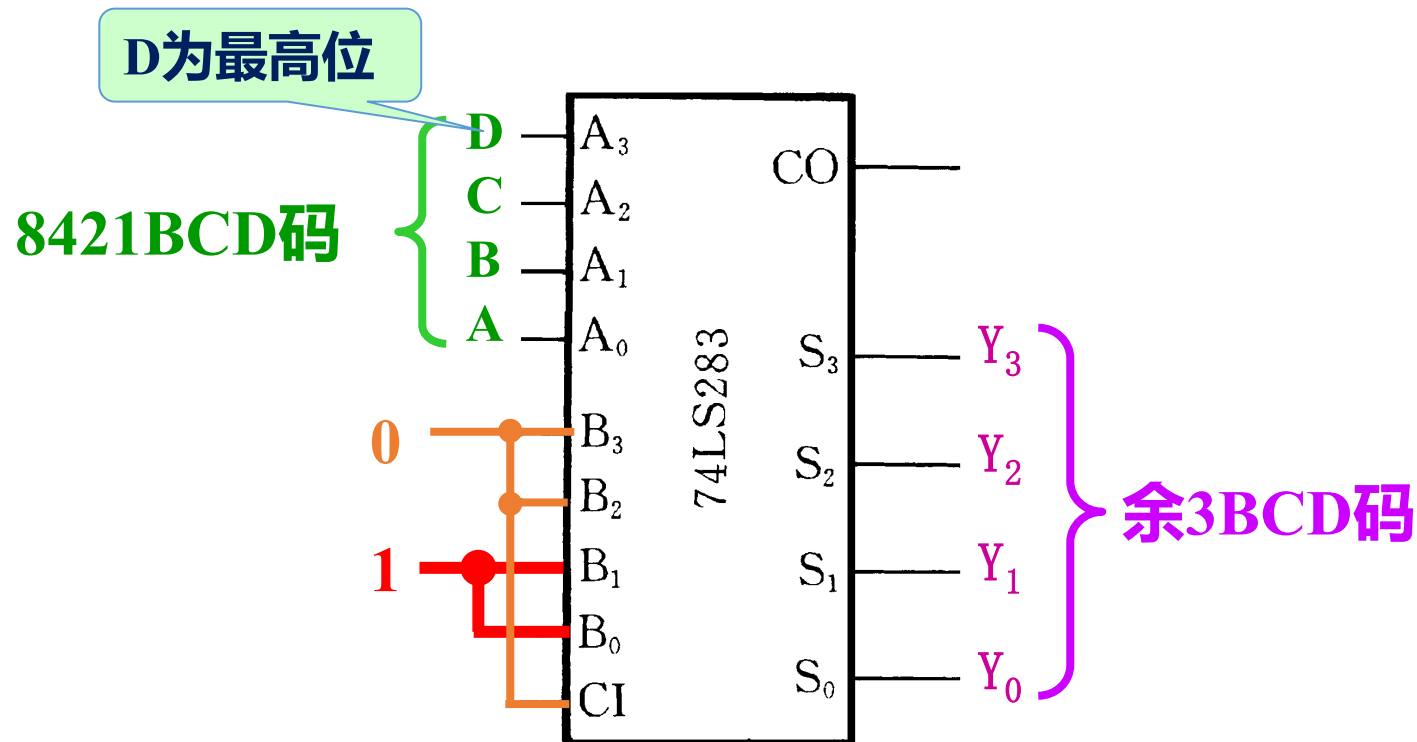
DIP16封装

3.5.7 运算电路(五)

(一)、加法器(续)

例3.5.12：用四位加法器实现8421BCD码至余3BCD码的转换。

余3BCD码比对应的8421BCD码多3，只需将输入的8421BCD码加3即可。



如果要实现的函数可以变换为两组输入变量之和或输入变量与常量之和的形式，用加法器来实现设计，往往比较简单。

3.5.7 运算电路(六)

(一)、加法器(续)

例3.5.13：试将两位8421BCD码转换为等值的二进制数。

设两位8421BCD码N的高位为 $D'_8D'_4D'_2D'_1$ ，低位为 $D_8D_4D_2D_1$

$$N = (D'_8D'_4D'_2D'_1D_8D_4D_2D_1)_{8421BCD} = (S_6S_5S_4S_3S_2S_1S_0)_2$$

$$= (D'_8 \times 8 + D'_4 \times 4 + D'_2 \times 2 + D'_1) \times 10 \\ + D_8 \times 8 + D_4 \times 4 + D_2 \times 2 + D_1$$

$$= D'_8 \times 80 + D'_4 \times 40 + D'_2 \times 20 + D'_1 \times 10 \\ + D_8 \times 8 + D_4 \times 4 + D_2 \times 2 + D_1$$

3.5.7 运算电路(七)

(一)、加法器(续)

$$= D_8' \times (64 + 16) + D_4' \times (32 + 8) + D_2' \times (16 + 4) + D_1' \times (8 + 2) \\ + D_8 \times 8 + D_4 \times 4 + D_2 \times 2 + D_1$$

$$= D_8' \times (2^6 + 2^4) + D_4' \times (2^5 + 2^3) + D_2' \times (2^4 + 2^2) + D_1' \times (2^3 + 2) \\ + D_8 \times 2^3 + D_4 \times 2^2 + D_2 \times 2 + D_1$$

$$= D_8' \times 2^6 + D_4' \times 2^5 + (D_8' + D_2') \times 2^4 + (D_4' + D_1' + D_8) \times 2^3 \\ + (D_2' + D_4) \times 2^2 + (D_1' + D_2) \times 2 + D_1$$

$$(S_6 S_5 S_4 S_3 S_2 S_1 S_0)_2 = S_6 \times 2^6 + S_5 \times 2^5 + S_4 \times 2^4 + S_3 \times 2^3 + S_2 \times 2^2 + S_1 \times 2^1 + S_0 \times 2^0$$

$$S_6 = D_8'$$

$$S_5 = D_4'$$

$$S_4 = D_8' + D_2'$$

$$S_3 = D_4' + D_1' + D_8$$

$$S_2 = D_2' + D_4$$

$$S_1 = D_1' + D_2$$

$$S_0 = D_1$$

3.5.7 运算电路(八)

(一)、加法器(续)

$$S_6 = D'_8$$

$$S_5 = D'_4$$

$$S_4 = D'_8 + D'_2$$

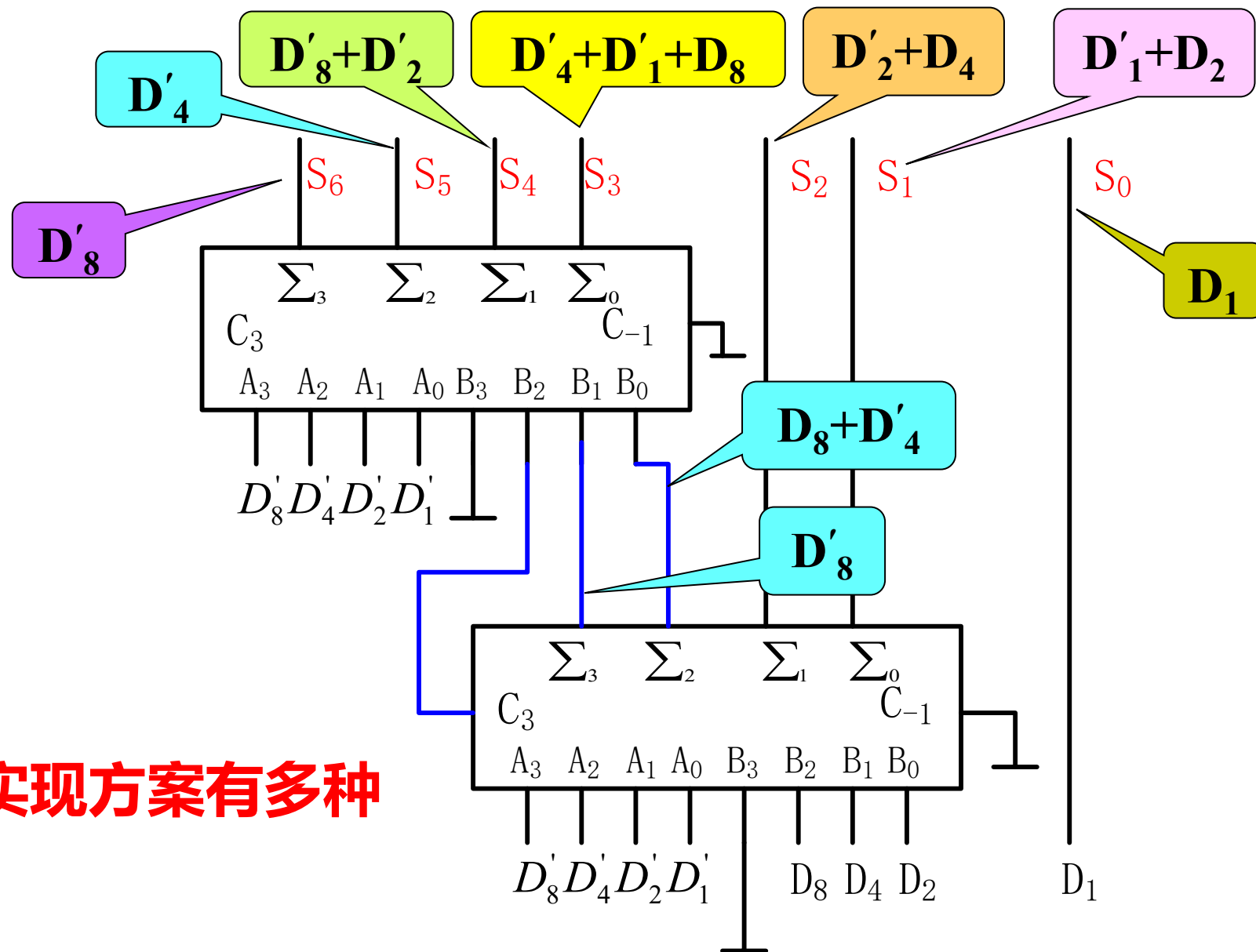
$$S_3 = D'_4 + D'_1 + D_8$$

$$S_2 = D'_2 + D_4$$

$$S_1 = D'_1 + D_2$$

$$S_0 = D_1$$

实现方案有多种



3.5.7 运算电路(九)

(二)、减法器

(1) 全减器

$$D_i = A_i \oplus B_i \oplus C_{i-1}$$

$$C_i = \overline{A_i}(B_i \oplus C_{i-1}) + B_i C_{i-1}$$

以 A_i 的反变量为输入

利用公式:

$$\overline{A \oplus B} = \overline{A} \oplus B = \overline{A} \cdot \overline{B} + AB$$

$$D_i = \overline{\overline{A_i} \oplus B_i \oplus C_{i-1}}$$

A_i	B_i	C_{i-1}	D_i	C_i
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

3.5.7 运算电路(十)

(二)、减法器(续)

全减器表达式

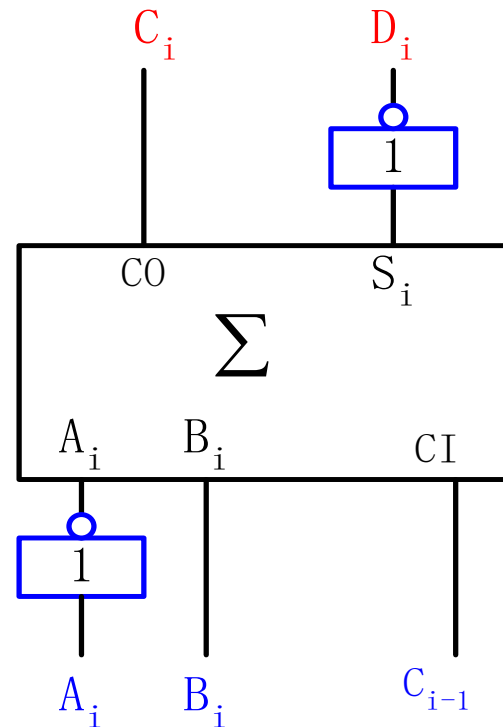
$$D_i = \overline{A_i \oplus B_i \oplus C_{i-1}}$$

$$C_i = (\overline{A_i} \oplus B_i)C_{i-1} + \overline{A_i}B_i$$

全加器表达式

$$S_i = A_i \oplus B_i \oplus C_{i-1}$$

$$C_i = (A_i \oplus B_i)C_{i-1} + A_iB_i$$



全减器

3.5.7 运算电路(十一)

(二)、减法器(续)

(2) 二进制正、负数的表示法

有符号二进制数：一位符号位 + 多位数值位

三种表示法：原码、反码和补码

原码：以最高位作为符号位，正数为0，负数为1，后面各位0和1表示数值

例： $(+9)_{10} = (01001)_2$ 原码

例： $(-9)_{10} = (11001)_2$ 原码

3.5.7 运算电路(十二)

(二)、减法器(续)

	原码	反码	补码
正数	符号位=0	与原码相同	与原码相同
	$(+9)_{10} = (01001)_2$	$(+9)_{10} = (01001)_2$	$(+9)_{10} = (01001)_2$
负数	$(-9)_{10} = (11001)_2$	$(-9)_{10} = (10110)_2$	$(-9)_{10} = (10111)_2$
	符号位=1	符号位=1 数值位按位取反	符号位=1 数值位按位求反, 最低位加1

负数在计算机中以补码的形式出现

补充：有符号二进制数

■ 无符号二进制数

- 特点：无符号位

$$A_{n-1}A_{n-2} \dots A_1A_0$$

表示范围：0 ~ 2^n-1

■ 有符号二进制数

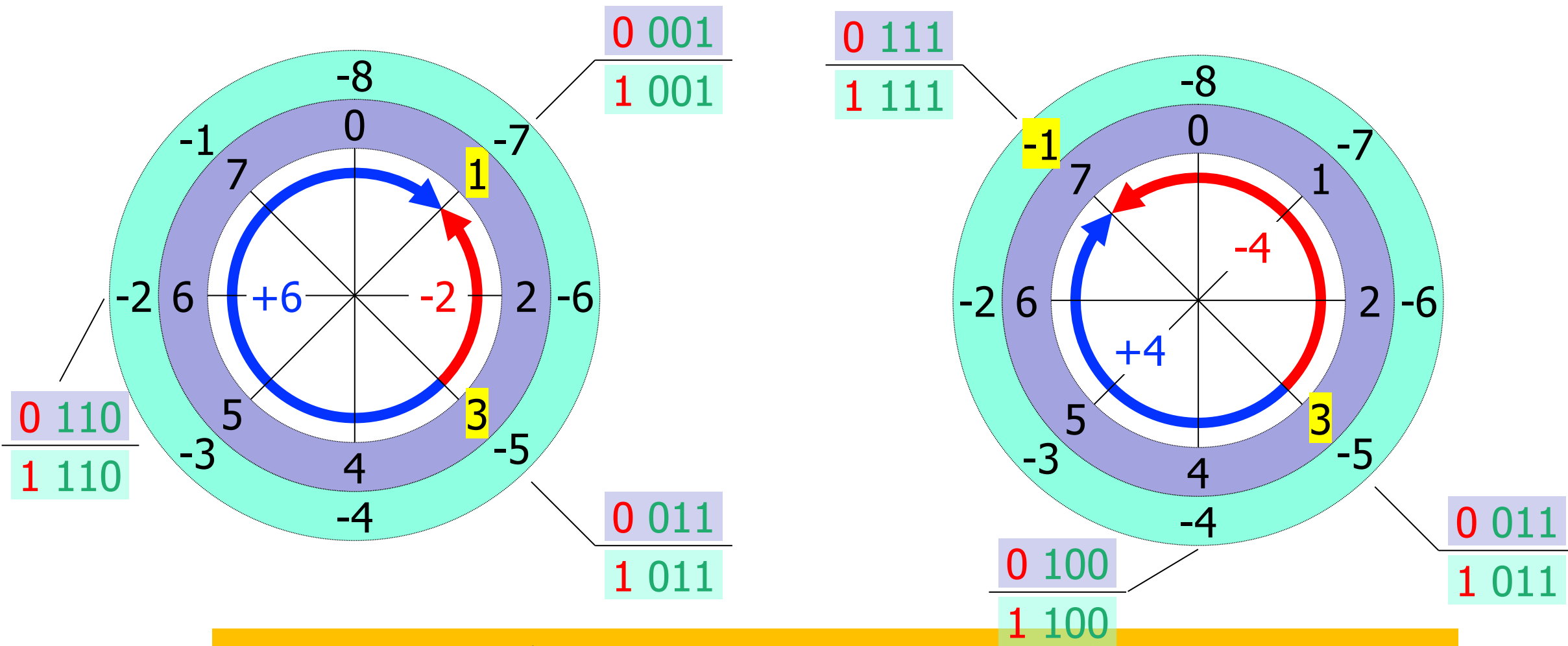
- 原码
- 反码
- 补码

符号位	数值部分					
-----	------	--	--	--	--	--

-2^{n-1}	2^{n-2}	...	...	2^2	2^1	2^0
------------	-----------	-----	-----	-------	-------	-------

8 位有符号数的最高位表示符号，0 表示正数，1 表示负数，其取值范围为：
-127 ~ +127 或 -128 ~ +127

补充：怎么理解补码的设计思路？



将减法转换成加法，一致性

3.5.7 运算电路(十三)

(二)、减法器(续)

(3) 补码表示法的减法

负数补码的补(连同符号位) = 其对应的正数

例: $(+18)_{10} = (010010)_2$

$(-18)_{10} = (101110)_{2\text{补码}}$ 再取补: $(010001)_2 + (1)_2 = (010010)_2 = (+18)_{10}$

$$(\pm A) - (+B) = (\pm A) + (-B)$$

将被减数和减数取补,
将减法变为补码加法

$$(\pm A) - (-B) = (\pm A) + (+B)$$

负数的补码为正数,
减负数等于加正数

3.5.7 运算电路(十四)

(二)、减法器(续)

$$(1001)_2 - (0101)_2 = (0100)_2$$

$$\begin{array}{r} 1001 \\ - 0101 \\ \hline 0100 \end{array}$$

用补码运算:

例: $[1001]_{\text{补码}} = 01001$

例: $[-0101]_{\text{补码}} = 11011$

进位=1, 够减

$$\begin{array}{r} 01001 \\ + 11011 \\ \hline 100100 \end{array}$$

用补码运算, 将减法运算转化成了加法运算

3.5.7 运算电路(十五)

(二)、减法器(续)

例: $18-18=0$

$(+18)_{10} = (010010)_{2\text{补码}}$

$(-18)_{10} = (101110)_{2\text{补码}}$

进位=1, 够减

0 1 0 0 1 0

+ 1 0 1 1 1 0

1 0 0 0 0 0

例: $18-19=-1$

$(+18)_{10} = (010010)_{2\text{补码}}$

$(-19)_{10} = (110011)_{2\text{原码}}$

$(-19)_{10} = (101101)_{2\text{补码}}$

进位=0, 不够减, 结果为负数的补码

0 1 0 0 1 0

+ 1 0 1 1 0 1

0 1 1 1 1 1

1 0 0 0 0 1

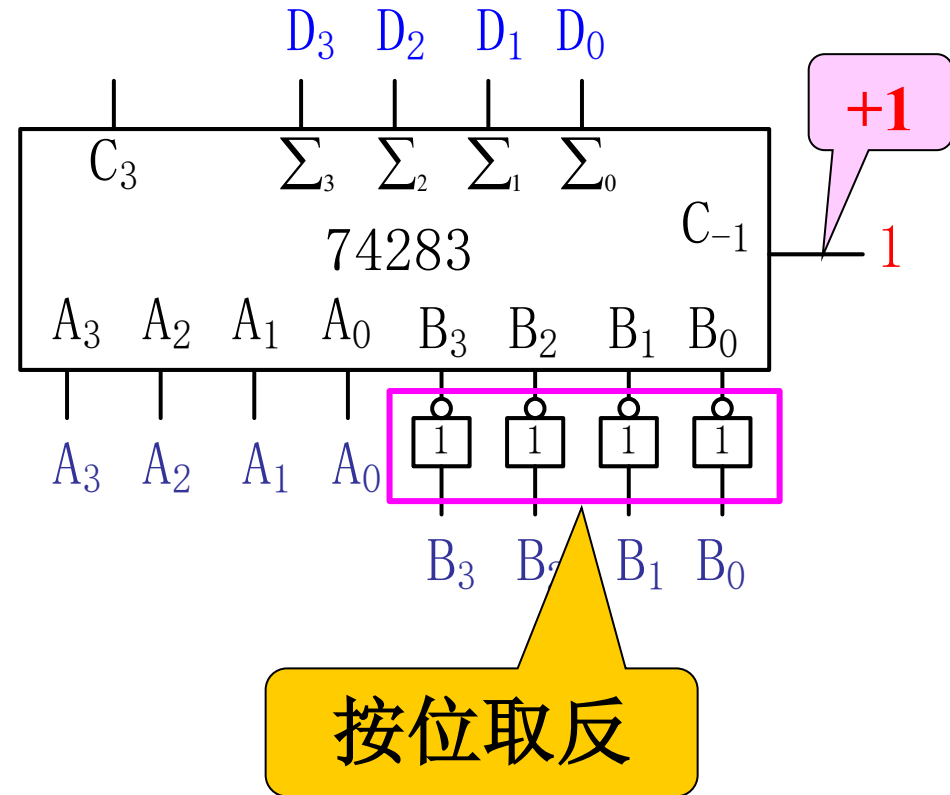
3.5.7 运算电路(十六)

(二)、减法器(续)

例：两个四位正数相减 $(+A) - (+B) = (+A) + (-B)$

$C_3=1$ ，够减， $D_3 \sim D_0$ 为差值正数。

$C_3=0$ ，不够减，输出 $D_3 \sim D_0$ 为负数补码，将其取补即为差的绝对值。



3.5.7 运算电路(十七)

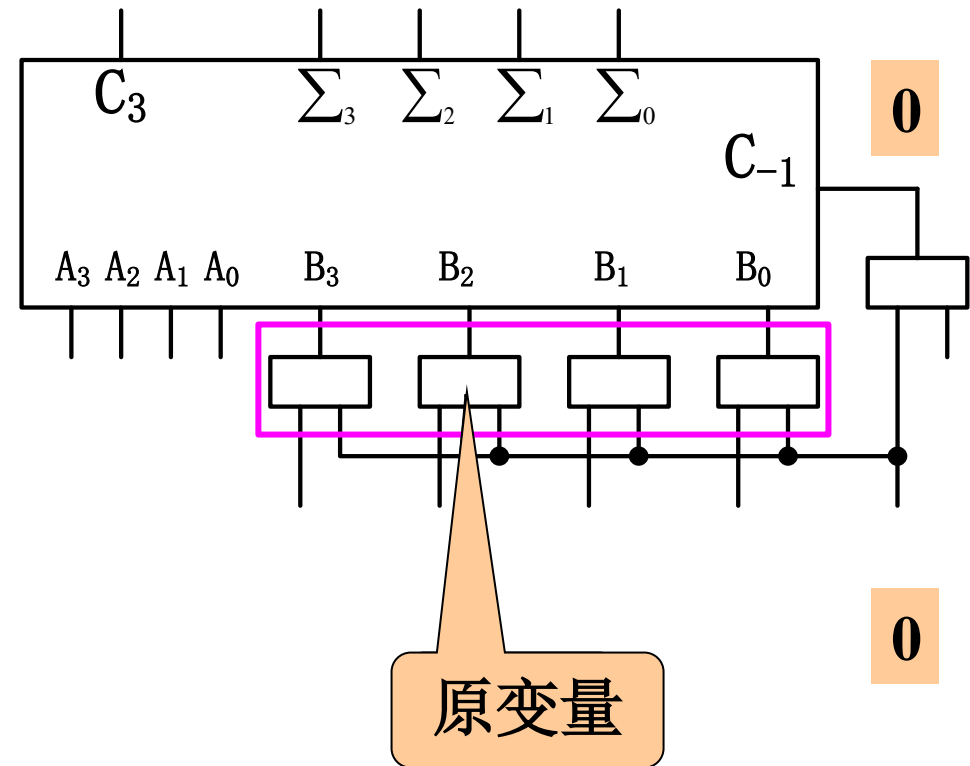
(二)、减法器(续)

可控加减电路

$$M \oplus B = M\bar{B} + \bar{M}B = \begin{cases} B & (M=0) \\ \bar{B} & (M=1) \end{cases}$$

M=1时，实现A-B运算

M=0时，实现A+B运算

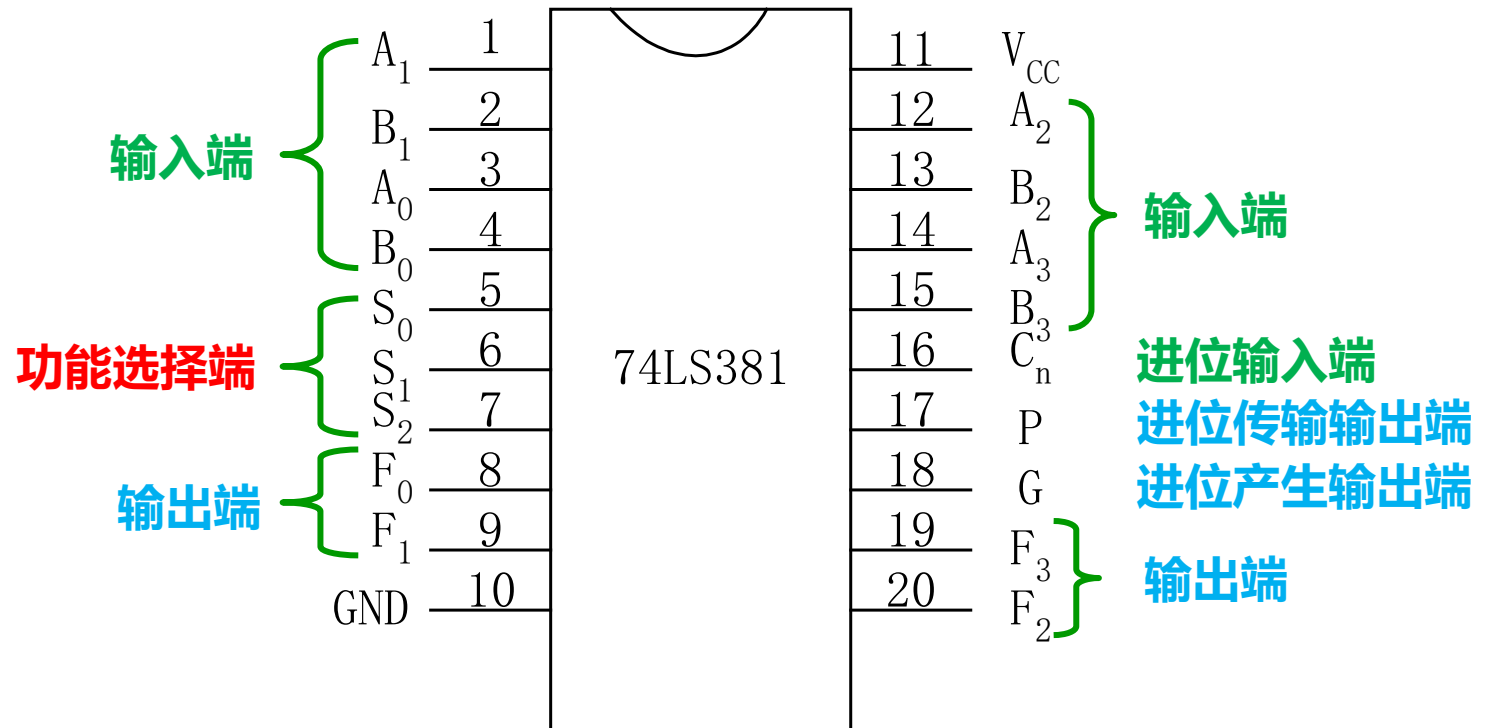


3.5.7 运算电路(十八)

(三)、算数逻辑单元(ALU)

能够执行数值比较、加法和减法等算术运算，与、与非、或、或非、异或和移位等逻辑运算以及逻辑运算和算术运算的混合运算的电路，也称**多功能函数发生器**

选择			算术/逻辑 操作
S_2	S_1	S_0	
0	0	0	清零
0	0	1	B-A
0	1	0	A-B
0	1	1	A+B
1	0	0	$A \oplus B$
1	0	1	A或B
1	1	0	$A \cdot B$
1	1	1	预置



3.5.7 运算电路(十九)

(三)、算数逻辑单元(ALU) 用MSI设计组合电路的步骤

- 1、进行逻辑抽象
- 2、写出逻辑函数式
- 3、将逻辑函数式变化为与所选器件的输出逻辑函数式类似形式，并进行比较
 - A、两者形式完全相同，直接按要求使用器件
 - B、两者形式类同，所选器件的输出逻辑函数式包含更多的输入变量和乘积项，需要对多余的输入端和乘积项作适当的处理
 - C、所选器件的输出逻辑函数式是所要求产生的逻辑函数的一部分，需要将几片器件联用或附加少量其它器件
 - D、两者形式相差甚远，就不宜采用所选器件来实现所要求的逻辑函数
- 4、画出逻辑图

本章小结(一)

(1) 组合电路

任何时刻的输出仅决定于**当时的输入**，而与电路原来的状态无关。它**由基本门构成**，**不含存储电路和记忆元件**，且**无反馈**

(2) 组合电路的分析

根据已经给定的逻辑图，描述其逻辑功能

(3) 小规模组合逻辑电路的设计

根据设计要求构成功能正确、经济、可靠的电路

本章小结(二)

(4) 组合电路的冒险

组合电路的静态逻辑冒险和功能冒险及产生原因和消除方法,判断电路是否存在冒险的方法

(5) 常用的中规模组合逻辑电路

用比较器、译码器、编码器、数据选择器、加法器和数据分配器等设计特定电路

用译码器、数据选择器和数据分配器可以实现任意逻辑函数